מערכת: פלטפורמת משלוחי אוכל (Wolt)

תיאור כללי: מערכת להזמנת משלוחי אוכל ממסעדות. משתמשים (לקוחות) מדפדפים בין מסעדות ותפריטים, מוסיפים מנות לעגלה ומבצעים הזמנה. לאחר מכן, שליחים אוספים את ההזמנה מהמסעדה ומביאים אותה ללקוח. התשלום מתבצע דרך המערכת, ובנוסף קיימת אפשרות לדירוגים וביקורות.

רצף עבודה (שחקנים):

- לקוח: נרשם באמצעות פרטים אישיים, שומר כתובות, ומשלם על הזמנות.
- מסעדה: מנהלת תפריטים, זמינות ומקבלת הזמנות.
- שליח: מקבל שיבוץ להזמנות, אוסף ומוסר אותן.

דרישותֵ:

- לכל לקוח חייבים להיות פרטי יצירת קשר וכתובת עדכנית למשלוח.
- ניתן לשמור מספר כתובות ללקוח אחדֵ.
- לכל מסעדה מוגדר אזור משלוח, ניתן להזמין ממנה רק אם הלקוח נמצא באזור זה.
- לכל מסעדה יש תיעוד של סוג מטבח, שעות פעילות, סטטוס (פתוח/סגור) ודמי משלוח בסיסיים.
- מסעדה יכולה להציע מספר בלתי מוגבל של מנות; לכל מנה יישמר שם, תיאור, מחיר וקטגוריית תפריט.
- לקוח יכול להזמין עד 5 יחידות מאותה מנה בהזמנה אחתֵ.
- לכל הזמנה נשמרים פרטי הלקוח, פרטי המסעדה, כתובת המסירה, סטטוס ההזמנה, דמי משלוח, עמלת פלטפורמה והסכום הסופי.
- מחיר המנה נשמר תחת ההזמנה כדי לשמר את המחיר בזמן הרכישה, גם אם המחיר בתפריט השתנה לאחר מכן.
- ניתן לשנות פרטי מנה בתפריט רק על ידי המסעדה.
- לכל שליח יישמרו פרטי יצירת קשר עדכניים ואמצעי תחבורה רלוונטי.
- רק מסעדות בסטטוס "פתוח" יכולות לקבל הזמנות.
- לכל הזמנה פעילה ניתן לשבץ שליח אחד בלבד; לא ניתן לשבץ שליח להזמנה שבוטלה.
- הזמנה תימסר רק לאחר תשלום מוצלח.
- לקוח יכול לדרג מסעדה פעם אחת לכל הזמנה; הדירוג הוא בין 1 ל־5.

1. כיתבו קוד ליצירת הטבלאות הנדרשות. הקפידו על נרמול.
2. פרטו נמקו את כל ההחלטות העיצוביות שקיבלתם:
   a. ציינו מה כל הישויות שאתם מייצגים, מה התרחיש בשבילו הם נדרשים, ומה היצוג שנבחר. אם יש ישויות רלוונטיות שאתם לא מייצגים ציינו והסבירו למה.
   b. הסבירו את השדות שבחרתם, התרחיש לו הם נדרשים, והטיפוס שבחרתם להם.
   c. נמקו את המפתחות שבחרתם, האם הם טבעיים (קיימים בעולם האמיתי) או פנימיים (לדוגמה auto increment)
   d. ציינו מה היחסים בין הישויות (1:1, 1:n, או n:n) בעולם האמיתי. אם בחרתם ליצג יחס אחר, ציינו ונמקו. מספיק לציין יחסים רק בין ישויות שכנות, שיש בניהן קשר ישיר.
   e. פרטו מה ה constraints (לדוגמה: unique, FK, not null, check) שבחרתם ומה ההגנה שהם מספקים. אם יש נקודות שלא מוגנות על ידי בסיס הנתונים ציינו אותן ונמקו איך להגן.
3. כיצד בחרתם לנהל את הסטטוס (פתוח/סגור). הסבירו כיצד בחרת להתמודד עם כך שלרוב יש שעות פתיחה קבועות אבל לעיתים יש בלת"מים.
4. האם בחרתם לאפשר לקוחות שונים באותה הכתובת? נמקו.
5. כיצד בחרתם להגדיר את איזור המשלוח? מרחק מהמסעדה, נסיון להתחשב זמן נסיעה, רשימת רחובות? נמקו
6. ציינו אפשרויות ליצוג שמות מנות ומנו יתרונות וחסרונות.
7. מה תציעו לוולט לעשות כאשר הם מציעים סוג חדש של מטבח? מדוע?
8. שילפו חריגות תמחור. מנה המוצעת בשתי מסעדות A ו B כך שהיא יקרה יותר ב A למרות שממוצע המחירים גבוה יותר ב B. חישבו היטב על שימוש ב views.

<u>תשובות</u>

בניית הטבלאות:

1. טבלת לקוחות

```
CREATE TABLE users (
  id         INT PRIMARY KEY AUTO_INCREMENT,
  full_name  VARCHAR(100) NOT NULL,
  email      VARCHAR(255) NOT NULL UNIQUE,
  phone      VARCHAR(30)
);
```

2. טבלת כתובות

```
CREATE TABLE addresses (
  Id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT NOT NULL,
  line1 VARCHAR(200) NOT NULL,
  Line2 text,
  city VARCHAR(100) NOT NULL,
  postal VARCHAR(20),
  region_id INT NOT NULL,
  CONSTRAINT fk_addr_user FOREIGN KEY (user_id) REFERENCES users(id)
```

```
  CONSTRAINT fk_addr_region FOREIGN KEY (region_id) REFERENCES regions(id)
);
```

3. טבלת מסעדות

```
CREATE TABLE restaurants (
  Id INT PRIMARY KEY AUTO_INCREMENT,
  name  VARCHAR(200) NOT NULL,
  cuisine_type VARCHAR(100) NOT NULL,
  status ENUM('OPEN','CLOSED') NOT NULL DEFAULT 'CLOSED'
  base_delivery_fee DECIMAL(6,2) NOT NULL
);
```

4. טבלת מנות

```
CREATE TABLE menu_items (
  id INT PRIMARY KEY AUTO_INCREMENT,
  restaurant_id INT NOT NULL,
  name VARCHAR(200) NOT NULL,
  description TEXT,
  price DECIMAL(8,2) NOT NULL,
  CONSTRAINT fk_item_rest FOREIGN KEY (restaurant_id) REFERENCES restaurants(id),
  CONSTRAINT u_rest_item UNIQUE (restaurant_id, name)  - אין אותה מנה פעמיים במסעדה
);
```

5. טבלת הזמנות

```
CREATE TABLE orders (
  id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT NOT NULL,
  restaurant_id INT NOT NULL,
  address_id INT NOT NULL,
  status ENUM('SUBMITTED', 'ACCEPTED','PREPARING' ,'OUT_FOR_DELIVERY',
'DELIVERED', 'CANCELLED') NOT NULL
  items_total   DECIMAL(8,2) NOT NULL,
  delivery_fee  DECIMAL(6,2) NOT NULL,
  platform_fee  DECIMAL(6,2) NOT NULL,
  total_amount  DECIMAL(8,2) NOT NULL,
  created_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
courier_id      INT NULL,
  delivery_status ENUM('ASSIGNED','PICKED_UP','DELIVERED','FAILED') NULL,
  assigned_at     DATETIME NULL,
  picked_up_at    DATETIME NULL,
  delivered_at    DATETIME NULL,
  CONSTRAINT fk_ord_user    FOREIGN KEY (user_id)       REFERENCES users(id),
  CONSTRAINT fk_ord_rest    FOREIGN KEY (restaurant_id) REFERENCES restaurants(id),
  CONSTRAINT fk_ord_addr    FOREIGN KEY (address_id)    REFERENCES addresses(id),
```

```
  CONSTRAINT fk_ord_courier FOREIGN KEY (courier_id)    REFERENCES couriers(id),
CONSTRAINT chk_fees_positive CHECK (delivery_fee > 0 AND platform_fee >= 0 AND
total_amount > 0)
);
```

6. **טבלת פריטי הזמנה**

```
CREATE TABLE order_items (
  id INT PRIMARY KEY AUTO_INCREMENT,
  order_id INT NOT NULL,
  menu_item_id INT NOT NULL,
  quantity INT NOT NULL,
  unit_price DECIMAL(8,2) NOT NULL,  -- מחיר בזמן ההזמנה
  CONSTRAINT fk_oi_order FOREIGN KEY (order_id) REFERENCES orders(id),
  CONSTRAINT fk_oi_item  FOREIGN KEY (menu_item_id) REFERENCES menu_items(id),
  CONSTRAINT chk_quantity CHECK (quantity BETWEEN 1 AND 5),
  CONSTRAINT u_item_once UNIQUE (order_id, menu_item_id)
);
```

7. טבלת שליחים

```
CREATE TABLE couriers (
  id INT PRIMARY KEY AUTO_INCREMENT,
  full_name VARCHAR(100) NOT NULL,
  phone VARCHAR(30) NOT NULL UNIQUE,
  vehicle   ENUM('BIKE', 'SCOOTER', 'CAR') NOT NULL
);
```

8. **טבלת שיבוץ שליחים להזמנה**

```
CREATE TABLE delivery_assignments (
  order_id INT PRIMARY KEY,
  courier_id INT NOT NULL,
  status ENUM('ASSIGNED','PICKED_UP','DELIVERED','FAILED') NOT NULL
  assigned_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_da_order   FOREIGN KEY (order_id) REFERENCES orders(id),
  CONSTRAINT fk_da_courier FOREIGN KEY (courier_id) REFERENCES couriers(id)
);
```

9. **טבלת ביקורות**

```
CREATE TABLE reviews (
  id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT NOT NULL,
  restaurant_id INT NOT NULL,
  order_id INT NOT NULL,
  rating INT NOT NULL,
  comment TEXT,
```

```
  created_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_rev_user  FOREIGN KEY (user_id) REFERENCES users(id),
  CONSTRAINT fk_rev_rest  FOREIGN KEY (restaurant_id) REFERENCES restaurants(id),
  CONSTRAINT fk_rev_order FOREIGN KEY (order_id) REFERENCES orders(id),
  CONSTRAINT chk_rating CHECK (rating BETWEEN 1 AND 5),
  CONSTRAINT u_review_once UNIQUE (user_id, order_id)  -- ביקורת אחת לכל הזמנה
);
```

10. <u>טבלת אזורים</u>

```
CREATE TABLE regions (
  id   INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL UNIQUE
);
```

11. <u>טבלת קשר בין אזורים לכתובת</u>

```
CREATE TABLE restaurant_regions (
  restaurant_id INT NOT NULL,
  region_id INT NOT NULL,
  PRIMARY KEY (restaurant_id, region_id),
  CONSTRAINT fk_rr_rest FOREIGN KEY (restaurant_id) REFERENCES restaurants(id),
  CONSTRAINT fk_rr_region FOREIGN KEY (region_id) REFERENCES regions(id)
);
```

<u>שאלה מספר 2:</u>

- **לקוחות (Users)** – מייצגים את האנשים שנרשמים לאפליקציה ומבצעים הזמנות. נחוצים כדי לזהות מי מבצע את ההזמנה ולשמור פרטי קשר.
- **כתובות (Addresses)** – מייצגות את כתובות המשלוח של הלקוחות. נחוצות כדי לאפשר לשלוח למספר כתובות שונות של אותו לקוח. בנוסף, שיוך לאזור משלוח (region) לכל כתובת.
- **מסעדות (Restaurants)** – מייצגות את המסעדות שמהן אפשר להזמין. דרושות כדי לדעת אילו מסעדות זמינות, איזה סוג מטבח הן מציעות ומה עלות המשלוח. base_delivery_fee מייצג את עלות המשלוח הבסיסית שגובה המסעדה או הפלטפורמה, לפני חישוב עמלות נוספות. שמתי אותו במסעדות כי לכל מסעדה יכולה להיות עלות משלוח שונה. זה מאפשר לחשב את עלות ההזמנה הסופית בצורה נכונה.
- **מנות (Menu items)** – מייצגות את פריטי התפריט של כל מסעדה. דרושות כדי שהלקוח יוכל לבחור אילו מנות להזמין ובאיזה מחיר.
- **הזמנות (Orders) –** מייצגות את פעולת ההזמנה עצמה: לקוח שבחר מסעדה וכתובת ומבצע הזמנה עם סכום ותאריך.
    - base_delivery_fee בטבלת המסעדות הוא ערך כללי של מסעדה, המשמש נקודת התחלה לחישובץ

- delivery_fee נשמר בכל הזמנה כדי לשמר את הערך בפועל בזמן ההזמנה (גם אם השתנה אחר כך).
- הקשר הוא לוגי: delivery_fee ≥ base_delivery_fee, אלא אם יש הנחה/מבצע.
- בחרתי לאחד את השדות הקשורים למשלוח ישירות לטבלת ההזמנות, משום שבמודל שלנו היחס בין הזמנה למשלוח הוא 1:1. אין צורך בהיסטוריית שיבוצים או בהחלפת שליחים, ולכן טבלה נפרדת מיותרת ומסבכת את המודל. השדות courier_id, delivery_status ותאריכי המשלוח מאפשרים לעקוב אחרי מצב המשלוח בצורה פשוטה וברורה.

- **פריטי הזמנה (Order items)** – מייצגים את הפירוט בתוך הזמנה: אילו מנות נכללו, בכמה יחידות ובאיזה מחיר.
- **שליחים (Couriers)** – מייצגים את האנשים שמביאים את ההזמנה ללקוח. דרושים כדי לעקוב אחר אספקת המשלוחים.
- **ביקורות (Reviews)** – מייצג את הדירוגים והתגובות של לקוחות על מסעדות אחרי שהזמינו. נחוץ כדי לאפשר משוב ושיפור השירות.
- **אזורי משלוח (Regions)** – מייצגים חלוקה גיאוגרפית פשוטה (למשל שכונה/אזור/רובע). נדרש כדי לקבוע לאיזה אזורים כל מסעדה מספקת.
- **שיוך מסעדות–אזורים (Restaurant regions)** – טבלת קשר שמייצגת אילו אזורים כל מסעדה מכסה (מסעדה אחת יכולה לכסות כמה אזורים, ואזור אחד יכול להיות מכוסה ע״י כמה מסעדות). בצורה זו כל מסעדה מגדירה מראש את האזורים שבהם היא פעילה, וכך כאשר לקוח בוחר כתובת מסוימת (עם שיוך לאזור), המערכת יודעת להציג לו רק את המסעדות שפועלות באותו אזור.

ישויות ללא ייצוג: שעות פתיחה, תשלומים/החזרים. הסיבה: לשמור את המודל לפי דרישות הלקוח (המסעדות). במודל לא ייצגתי מי מקבל בפועל את הכסף, משום שזה נושא פיננסי שלא היה חלק מהדרישות. הבחירה נעשתה במודע להתמקד בצד הלוגיסטי והתפעולי של ההזמנה.

<u>בחירת שדות:</u>

- מזהים: INT AUTO_INCREMENT PRIMARY KEY – מפתח פנימי פשוט ויציב לתרגיל.
- טקסטים: VARCHAR לשמות/אימייל/עיר; TEXT לתיאורי מנות/תגובות.
- כספים: DECIMAL למחירים/עמלות – התאמה למטבעות.
- זמנים: DATETIME לשדות יצירה/שיוך – מאפשר היסטוריית הזמנות ושיבוצים.
- סטטוסים/דירוגים: ENUM כדי להגביל לערכים חוקיים בלבד (פתוח/סגור; סטטוס הזמנה; סטטוס שיבוץ; דירוג 1–5).
- כמות פריטים: quantity INT בשורת פריט אחת לכל מנה (כמות מייצגת כמה הוזמן).
- שדות לאזורי משלוח: לכל כתובת נוסף region_id; בטבלת אזורים נשמר name ייחודי; בטבלת שיוך מסעדות–אזורים נשמר זוג (restaurant_id,region_id).

<u>מפתחות:</u>

- Primary Keys: בכל הטבלאות INT AUTO_INCREMENT (או order_id כ־PK ב־delivery_assignments. פשוטים לקישור בין טבלאות.

- ייחודיות היכן שמתבקש:
  - users.email UNIQUE – אי־כפילות משתמשים.
  - couriers.phone UNIQUE – אי־כפילות שליחים לפי טלפון.
  - menu_items(restaurant_id,name) UNIQUE – לא אותה מנה באותה מסעדה.

- reviews(user_id,order_id) UNIQUE – ביקורת אחת של אותו לקוח לאותה הזמנה.
- regions.name UNIQUE – מניעת כפילות שמות אזורים.
- restaurant_id,region_id כ־PRIMARY KEY משותף – מונע כפילות שיוכים.
- addresses.region_id כ־FOREIGN KEY ל־regions – מחייב שיוך כל כתובת לאזור קיים.
- restaurant_id,region_id כ־FOREIGN KEY ל־restaurants ו־regions בטבלאת הקישור.

4) יחסים בין ישויות

- ללקוח אחד יכולות להיות מספר כתובות: יחס של אחד לרבים. במודל הנוכחי כל כתובת שייכת ללקוח אחד, אבל במציאות ייתכן ששותפים יגורו יחד. אפשר לפתור זאת ע"י שיוך כתובת ליותר מלקוח אחד (יחס רבים־לרבים דרך טבלת שיוך), אך לשם פשטות בחרתי במודל שבו כל כתובת מוגדרת בנפרד לכל לקוח.
- למסעדה אחת יכולות להיות מספר מנות בתפריט: יחס של אחד לרבים.
- להזמנה אחת יכולים להיות מספר פריטי הזמנה (מנות): יחס של אחד לרבים.
- כל הזמנה שייכת ללקוח אחד, למסעדה אחת ולכתובת אחת: יחס של רבים לאחד.
- לכל הזמנה יכול להיות לכל היותר שיבוץ שליח אחד: יחס של אחד לאחד.
- כל לקוח יכול לכתוב מספר ביקורות לאורך הזמן, אך לכל הזמנה מותרת רק ביקורת אחת ממנו: יחס של אחד לרבים עם מגבלה מיוחדת (ביקורת אחת להזמנה).
- כל כתובת שייכת לאזור אחד: יחס של רבים לאחד.
- מסעדה יכולה להיות משויכת למספר אזורים, ואזור יכול להיות משויך למספר מסעדות: יחס של רבים לרבים (דרך טבלת השיוך).

התאמה לעולם: משקף נכונה את המציאות.

5) Constraints שנבחרו ועל מה הן מגנות:

- **חובה למלא שדות מסוימים**: למשל שם מלא או מחיר – מוודא שלא יישמרו רשומות חסרות.
- **ייחודיות** – כמו באימייל של לקוח או במספר טלפון של שליח – מונעת כפילויות לא הגיוניות.
- **מפתחות זרים** – מקשרים בין טבלאות, ומבטיחים שלא תהיה הזמנה ללקוח או למסעדה שלא קיימים.
- **ערכים מוגבלים מראש** – שדות סטטוס ודירוג הוגדרו כך שניתן להזין רק ערכים מוגדרים (לדוגמה דירוג מ־1 עד 5).
- **בדיקת טווחים** – על כמות מנות בהזמנה, כדי לאפשר עד 5 יחידות בלבד.
- **עבור אזורי משלוח:**
  - addresses.region_id כ־FK ל־regions – כל כתובת שייכת לאזור נכון.
  - restaurant_regions עם PK מורכב ו־FKs – מונע שיוכי כפילות ומבטיח עקביות בין מסעדות לאזורים.

מה שלא מוגן ברמת מסד הנתונים: בדיקה אם מסעדה פתוחה באותו רגע לפי שעות פעילות.

ניהול סטטוס (פתוח/סגור):

- במודל בחרתי שדה סטטוס פשוט במסעדה: פתוח/סגור. זה מאפשר שליטה תפעולית ידנית ומהירה.
- לגבי שעות פתיחה קבועות: לא ייצגנו שעות בטבלאות; ההנחה היא שהאפליקציה תבדוק "האם עכשיו שעות הפעילות" על בסיס תצורה חיצונית או הרחבה עתידית.

- לגבי בלת"מים (סגירה זמנית, חופשה, תקלה): הטיפול הוא באמצעות אותו שדה סטטוס, מעבירים ל"סגור" כשהמסעדה לא זמינה.

<u>לקוחות שונים באותה כתובת</u>

- כן. בחרתי לאפשר שלכמה לקוחות תהיה אותה כתובת (משפחה, דירה משותפת, משרד). לכן אין אילוץ ייחודיות על טקסט הכתובת.
- הכתובת שייכת ללקוח כדי לנהל ספר כתובות אישי (כינויים, הערות לדלת, ברירת מחדל וכד').
- אם בעתיד נרצה לזהות כתובות זהות לצורך ניתוח, אפשר להשוות טקסטואלית או להוסיף מזהי כתובת תקניים.

<u>הגדרת אזור המשלוח:</u>

בחרתי להגדיר אזור משלוח כיישות נפרדת בשם **Region**, המייצגת חלוקה גיאוגרפית פשוטה (למשל שכונה, רובע או עיר). לכל כתובת לקוח משויך אזור אחד, ולכל מסעדה מוגדרים האזורים שאליהם היא מספקת באמצעות טבלת קשר (restaurant_regions).

נימוק:

- שימוש במרחק גיאוגרפי או חישובי זמן נסיעה דורש חישובים בזמן אמת ומורכב יותר מבחינת ביצועים.
- רשימת רחובות עלולה להפוך למסורבלת לניהול ולדרוש עדכונים תכופים.
- חלוקה לפי אזורים (שכונה/עיר) היא פשרה נוחה: פשוטה לתחזוקה, מובנת למשתמשים, ויעילה טכנית.

<u>אפשרויות לייצוג שמות מנות:</u>

1. **טקסט חופשי (VARCHAR / TEXT)**
    - *יתרון:* גמישות מלאה – המסעדה יכולה לרשום כל שם שתרצה, כולל גרסאות חדשות.
    - *חיסרון:* עלולות להיות כפילויות וקשה לבצע חיפושים או ניתוחים מדויקים.
2. **בחירה מתוך רשימה מוגדרת מראש (לדוגמה ENUM או טבלת מנות אחידה)**
    - *יתרון:* מבטיח אחידות, נוח להשוואות בין מסעדות (לדוגמה השוואת מחירים לאותה מנה).
    - *חיסרון:* מגביל את המסעדות, קשה להוסיף שמות חדשים ללא שינוי במבנה הנתונים.
3. **שילוב: טבלת מנות מרכזית + שדה טקסט חופשי למסעדה**
    - *יתרון:* מאפשר גם אחידות (זיהוי שמות כלליים) וגם חופש למסעדות להוסיף גרסאות ייחודיות.
    - *חיסרון:* המודל מורכב יותר ודורש תחזוקה כפולה.

<u>כאשר וולט מציעים סוג חדש של מטבח:</u>
המערכת בנויה כך שסוגי המטבח נשמרים כטבלה נפרדת (Cuisines) ולא כרשימה סגורה. כאשר נוסף סוג מטבח חדש, וולט לא יכולה פשוט להכניס אותו למסעדות קיימות, כי רק המסעדות יודעות אם הן אכן מציעות את

הסוג הזה. לכן התהליך הוא: וולט מוסיפה את הסוג למילון סוגי המטבח, ואז פונה למסעדות הרלוונטיות שיבחרו אם לשייך עצמן אליו. זה מונע מצב שבו מסעדות מקבלות שיוך לא נכון.

נימוק:

- זה מאפשר גמישות של המערכת.
- אין צורך בשינויים בקוד או בטבלאות אחרות - רק הוספה של ערך חדש.
- נמנעת מגבלה של רשימה (כמו ENUM), שמחייבת שינוי סכמה בכל פעם.

<u>חריגות תמחור</u>

```
CREATE VIEW v_price_anomalies AS
SELECT
  a.restaurant_id AS rest_a,
  b.restaurant_id AS rest_b,
  a.name AS item_name,
  a.price AS price_a,
  b.price AS price_b,
  avgA.avg_price AS avg_price_a,
  avgB.avg_price AS avg_price_b
FROM menu_items a
JOIN menu_items b
  ON b.restaurant_id <> a.restaurant_id
 AND LOWER(TRIM(b.name)) = LOWER(TRIM(a.name))
JOIN (SELECT restaurant_id, AVG(price) AS avg_price
      FROM menu_items GROUP BY restaurant_id) avgA
  ON avgA.restaurant_id = a.restaurant_id
JOIN (SELECT restaurant_id, AVG(price) AS avg_price
      FROM menu_items GROUP BY restaurant_id) avgB
  ON avgB.restaurant_id = b.restaurant_id
WHERE a.price > b.price AND avgB.avg_price > avgA.avg_price;
```